

Real-Time Flocking Simulation with Obstacle Avoidance

Duong Vu

December 11, 2025

1 Introduction and Motivation

Large-scale coordinated motion appears frequently in nature, most notably in flocks of birds, schools of fish, and swarms of insects. These systems exhibit highly organized group behavior, despite the absence of any central controller. Instead, mesmerizing complex patterns emerge from simple, local interaction rules followed by each agent. Understanding and reproducing this behavior has long been a classic problem in computer graphics and simulation, as it shows how emergent behavior can arise from simple local interactions.

This project is motivated by both theoretical and practical interest in real-time simulation. Through prior coursework and experience as a physics student, I have encountered flocking as a topic in biophysics and emergent behavior, prompting me to wonder how such systems can be modeled computationally. Additionally, exposure to the interactive online flocking demo in class highlighted how fluid and natural these simulations are, prompting me to wonder about the underlying algorithm and implementation techniques. By building a real-time flocking simulation with obstacle avoidance, this project extends the classic Boids framework to more complex and realistic environments, examining how collective motion adapts when navigating and avoiding obstacles in the scene.

2 Background work

This project builds on the foundational work of Craig Reynolds' Boid algorithm [2], which demonstrated that such emergent flocking behavior can be recreated using three simple local steering forces. That is, each boid follows these three simple local rules

- **Separation:** Avoid flying into nearby flockmates
- **Alignment:** Match velocity with nearby flockmates
- **Cohesion:** Fly close to the center of nearby flockmates

The specific implementations of these forces and the overall structure of the simulation are adapted from Daniel Shiffman's Processing "Flocking" tutorial [1], which provides a modern, interactive implementation of the Boids algorithm.

3 Methodology

The primary goal of this project is to recreate an interactive, real-time flocking simulation and extend it with obstacle avoidance. The system is implemented in Python using the Pygame library, which supports real-time visualization and user interaction. Obstacles can be efficiently added or removed, allowing exploration of how collective behavior adapts under different constraints. The implementation is organized around three main classes: **Boid** (the individual agent), **Wall** (the obstacle), and **Flock** (the simulation manager).

3.1 Boids Dynamics

Each boid maintains a kinematic state consisting of its position $\vec{x}$ and velocity $\vec{v}$. Each boid is constrained to have a maximum speed $v_{\max}$. For simplicity, we take the mass of each boid to be 1, so forces directly produce acceleration. Given a desired direction $\vec{d}$ for the boid to move toward, the steering force is computed as

$$\vec{F}_{\text{steer}}(\vec{d}) = \text{clamp}\left(v_{\max} \frac{\vec{d}}{\|\vec{d}\|} - \vec{v}, a_{\max}\right). \quad (1)$$

The clamp function limits the acceleration magnitude to $a_{\max}$. Importantly, this clamping is applied to each individual behavioral force rather than the net force, preventing any single component from dominating and producing smoother motion.

For flocking interactions, each boid considers neighbors within a radius r that depends on the behavior. Each flocking term is computed by first forming a desired direction $\vec{d}$, then converting it into a steering force via $\vec{F}_{\text{steer}}$. These interactions are given by

- **Separation** (radius: 50 units).

Neighboring boids exert a repulsive influence to prevent collisions, with closer boids having stronger effect:

$$\vec{d}_{\text{sep}} = \frac{1}{N} \sum_i \frac{\vec{x} - \vec{x}_i}{\|\vec{x} - \vec{x}_i\|^2}, \quad (2)$$

$$\vec{F}_{\text{sep}} = \vec{F}_{\text{steer}}(\vec{d}_{\text{sep}}). \quad (3)$$

- **Alignment** (radius: 100 units).

The boid aligns its motion with the average velocity of its neighbors:

$$\vec{d}_{\text{ali}} = \frac{1}{N} \sum_i \vec{v}_i, \quad (4)$$

$$\vec{F}_{\text{ali}} = \vec{F}_{\text{steer}}(\vec{d}_{\text{ali}}). \quad (5)$$

- **Cohesion** (radius: 100 units).

The boid steers toward the average neighbor position to remain with the group:

$$\vec{d}_{\text{coh}} = \left(\frac{1}{N} \sum_i \vec{x}_i\right) - \vec{x}, \quad (6)$$

$$\vec{F}_{\text{coh}} = \vec{F}_{\text{steer}}(\vec{d}_{\text{coh}}). \quad (7)$$

In the simulation, these forces are weighted by

$$w_{\text{sep}} = 1.5, \quad w_{\text{ali}} = 1.0, \quad w_{\text{coh}} = 1.0. \quad (8)$$

3.2 Obstacle Avoidance

Obstacle avoidance is implemented using predictive collision detection. The idea is similar to ray tracing: from the boid's current position, a ray is traced in the direction of its velocity,

$$\vec{x}(t) = \vec{x} + \hat{v} t, \quad (9)$$

where $\hat{v}$ is the normalized velocity. A `HitRecord` object is used to store the smallest value of t for which an intersection occurs, along with the corresponding surface normal. Each obstacle primitive implements a `hit` method that takes the ray origin, ray direction, and hit record and updates the record if a closer intersection is found.

The hit record is initialized with $t_{\text{max}} = 200$, so only intersections closer than this lookahead distance are considered. The ray is tested against all obstacles in the environment, producing the nearest intersection if one exists. If an intersection is detected, the boid computes an avoidance force based on the hit record. The desired direction is defined as

$$\vec{d}_{\text{obs}} = v_{\text{max}} \hat{n} + 0.5 v_{\text{max}} \hat{v}, \quad (10)$$

where $\hat{n}$ is the surface normal at the point of impact. The corresponding steering force is

$$\vec{F}_{\text{obs}} = \vec{F}_{\text{steer}}(\vec{d}_{\text{obs}}). \quad (11)$$

This direction is chosen so that

1. it pushes the boid away from the obstacle along the normal direction, and
2. it adds a tangential component proportional to the boid's current motion, allowing it to maintain forward momentum while sliding past the surface.

In the simulation, this force is scaled by a factor of 5.0 so that obstacle avoidance takes precedence over the flocking forces. As a result, boids navigate smoothly around obstacles while preserving group cohesion whenever possible. The simulation currently supports the following obstacle types:

- straight walls,
- circular walls,
- rectangular walls (including rotated ones),
- general polygonal walls.

It also supports moving circular walls, which is the most natural case since they have a well-defined center of motion.

3.3 Integration and Rendering

The simulation runs at 60 FPS using Pygame’s timing loop. Each frame, the `Flock` manager updates all boids and walls by calling their respective `run` and `update` methods. Each boid performs the following sequence of operations:

1. Computes flocking forces (separation, alignment, cohesion),
2. Computes obstacle-avoidance force,
3. Updates velocity and position via simple Euler integration,
4. Applies wraparound boundary conditions,
5. Renders itself as a velocity-oriented triangle.

The main loop also handles user interaction, allowing new boids to be spawned by left-clicking anywhere on the screen, enabling real-time control over the number of agents.

4 Results

The implemented system successfully reproduces the core behavior of boids and extends it to incorporate obstacle avoidance. We can see that

- **Flocking Behavior:**

With 250 boids initialized at the center of the screen, the simulation exhibits characteristic flocking dynamics. Boids quickly self-organize into a cohesive cluster that moves smoothly across the environment. Separation prevents collisions, alignment maintains consistent heading, and cohesion keeps the flock unified even as it stretches and reforms.

- **Obstacle Avoidance:**

The system robustly handles a wide variety of obstacle types. Boids steer around straight walls, orbit and slide past circular obstacles (including moving ones), and flow naturally through narrow gaps formed by complex polygonal shapes. The predictive ray-based lookahead produces anticipatory steering rather than abrupt last-second corrections, resulting in smooth trajectories around obstacles.

- **Scaling and Performance:**

The simulation maintains interactive frame rates with 250–300 boids while performing per-boid neighbor searches, force computation, obstacle intersection tests, and rendering each frame. This demonstrates that the implementation scales well for moderately large flocks while still supporting real-time interaction, such as spawning new boids during runtime.

5 Future Work

Several extensions could further enhance the system’s capabilities and visual richness. Spatial acceleration techniques, such as quadtree or grid-based partitioning, could reduce the computational cost

of neighbor-finding, allowing larger flock sizes. Predation and pursuit behaviors could be introduced by adding “predator” agents that chase boids and implementing “prey” behavior to flee, thereby creating more complex ecological interactions and dynamic scenarios. Visualization enhancements, such as rendering neighborhood zones, force vectors, and collision points, would provide educational insight into the simulation’s internal state.

References

- [1] Daniel Shiffman. Flocking. <https://processing.org/examples/flocking/>, n.d. Accessed: 2025-11-06.
- [2] Craig W. Reynolds. Flocks, herds and schools: A distributed behavioral model. *SIGGRAPH Comput. Graph.*, 21(4):25–34, August 1987.